

Random, Genesis, E0

Three builds this week. One proved the plumbing, one tested the way we work, and one ran the first real experiment — which came back worse than random. That last one is the interesting part.

Builds 3 **Submitted** 1 (scored 0.07) **GPU used** 1.35 h of 30 **Milestone 2** 30 September
Final 2 November

The week in three builds

Each build had one job. None of them was "make the agent good" — that comes after we can measure things honestly.

BUILD 1 · RANDOM

Prove the plumbing

An agent that picks legal moves at random. The point was never the score: it was to prove a notebook can install the game engine *with no internet*, play 110 hidden games, and produce a valid result file. Before this, none of that was tested.

Result: 0.07 out of 100 on the hidden games. The pipeline works.

BUILD 2 · GENESIS

Test the way we work

The same code taken through the Genesis harness. I wrote down four tests *before* running it, so the verdict couldn't be fitted to the outcome afterwards. All four passed. The one that mattered: after I changed a line of code, Genesis refused to mark the work finished until the checks were re-run.

Result: adopted. It later caught two of my own mistakes.

BUILD 3 · E0

Measure a real model

A frozen open-weight model (Gemma-4-E4B) playing 17 held-back games, five times each, through a deliberately minimal wrapper. This is the baseline every later idea gets compared against.

Result: below random on every measure. Diagnosis in progress.

How we work now

The same five steps every time. The order is the point: the promises come before the data, so results can't be reverse-engineered into successes.

01 Write it down first What we're testing, and what result would prove it wrong. Dated, committed.	02 Fix the unknowns Which games are held back, how many attempts, which settings — before any model plays.	03 Rehearse locally Run the whole thing on this laptop against the real engine, so Kaggle isn't where we discover bugs.
04 Gate, then run Automated checks must pass before anything is pushed. No check, no push.	05 Record, don't remember Decisions, failures and numbers go in the repo with their sources, not in chat.	

Step 1 is the habit worth stealing. "This experiment is wrong if X happens" turns an opinion into something that can fail. It is also what makes a negative result publishable instead of embarrassing.

What the numbers say

Everything is measured against a random agent given the exact same budget — same games, same number of attempts, same 80 moves per game.

MEASURE	RANDOM	GEMMA-4-E4B	WHAT IT MEANS
Levels cleared	0.04	0.00	Average per game, across the 16 games that count
Screens explored	78.7	67.5	Distinct screens per 100 moves — how much it changes the world
Games it beat random on	—	0 / 16	The headline: the model lost everywhere
Replies we couldn't read	—	0%	It answered correctly every time. It simply chose badly
Time per move	—	9.3 s	Each move sends ~20,000 tokens: the screen plus four past screens

Why a bad result is still a good week

We now have a floor with a number on it, measured the same way for every future idea. Before this, "does our agent help?" had no answer because there was nothing to compare to.

And we now know *why* it lost. Two guesses were wrong first: a policy that repeats one move, and one that flips between two moves, both explored *more* than the model did. A run that logged every move it chose gave the real answer.

Where the loss actually comes from

Some games are played with keys alone; others let you click a square. Split the games that way and the result is unambiguous.

TYPE OF GAME	GAMES	MODEL VS RANDOM	READING
Keyboard only	4	+0.3	Matches random. No problem here
Has a click action	13	-17.3	The entire deficit lives here

The logged moves show what it does: it clicks the **same dead squares over and over** — corners and edges like (0,0) and (63,9) — where nothing happens. On a keyboard-only game it spread its moves evenly and explored exactly as much as random did.

That is a fixable problem, and an honest one to report: the model isn't broken and our wrapper isn't broken. A 64×64 grid of digits gives it no good way to pick *where* to click.

Three things broke, and the process caught them

Worth showing peers: each was caught by a check rather than by a bad number in a report.

- 1 **A library refused Kaggle's newest GPU**, claiming it was too old — it isn't; the library misreads it. One setting fixed it. Found on a 7-minute test run, not on a 70-minute one.
- 2 **A path written the wrong way for Windows** made a check fail before it ran any code. The saved error output made it a one-read diagnosis.
- 3 **I broke a file with a bad edit**. The gate refused to let it be pushed. Without it, a broken notebook would have gone to Kaggle and wasted a run.

The repo, and what the rules allow

The code lives in a **private** GitHub repo. That isn't secrecy for its own sake — the competition rules are specific about sharing, and they're worth everyone knowing.

Rule 3.6

No private sharing of competition code outside your team. If you want to share it publicly, it has to go through Kaggle's own forums or notebooks, where everyone can see it. So: my harness code is not something I can pass around the cohort privately.

Rule 2.4.b

Don't redistribute the competition data. The 25 game files stay out of git entirely. They're ignored, and I removed them from the history before the

first push.

Rule 2.5.a	Prize winners must open-source everything — code, model and weights, under an approved open-source licence. That's why every model I shortlisted is Apache 2.0, and why a model with a custom licence was ruled out.
Teams	Up to 8 people, merge deadline 26 October. This is the clean way to collaborate: teammates can share code freely. If you want in, that's the mechanism.
Runtime	9 hours, no internet, GPU quota 30 h/week. The agent is on its own once submitted: no API calls, so the model has to be local.

What I can share freely

Everything about *method*: the literature review and its five notes, the experiment register, the way results are recorded, and general facts about the toolkit. Anyone can also get the official starter kit from ARC Prize and reproduce build 1 themselves in about ten minutes — that's public, and it's the fastest way to see the whole pipeline work.

Next, and where help lands

NEXT	WHY	COST
Fix how clicks are chosen	Diagnosis done: it clicks dead squares, and that is the whole gap	design first
Second small model (Qwen3-8B)	Completes the small-model comparison	~1 h GPU
Two big models (27–31B)	The sizes the June winners used, where scores are 5–11	~3–4 h each
E1: the real experiment	Does remembering failures and avoiding repeated states help?	after E0

Two dates

30 September — optional milestone prize, and it needs a public notebook on the leaderboard by then. **2 November** — final submission. Entry and team merges close 26 October.

Sixth note in the ARC Prize 2026 series, after the milestone brief, the hypothesis plan, *From 590 to 58*, *What the 58 Say* and *What Would Prove It Wrong*. Figures from this week's runs: Kaggle submission 56171083, kernels `arc-agi-3-e0-smoke`, `arc-agi-3-e0-gemma4-e4b`. Compiled 12 September 2026.